

Module 1 : Modélisation Objet Avancée

Python & Java

Classes & Objets

Héritage

Encapsulation

Polymorphisme

Évaluation : 40% TP · 60% Contrôle Continu

La Roadmap du Coursus : Ingénierie Logicielle

01

Modélisation Objet Avancée

Classes, Objets, Héritage,
Encapsulation, Polymorphisme

02

Principe SOLID & Qualité

5S, Principes SOLID

03

Patrons de Conception

Design Patterns GoF
(Création, Structure , Developpement)

04

Modélisation UML

Représentation visuelle
des architectures & Diagrammes

Objectif final : Concevoir · Implémenter · Tester · Agilité logicielle & Qualité industrielle

Programmation Classique vs Orientée Objet

Pourquoi la POO ? — Quel problème résout-elle par rapport à la programmation procédurale ?

Approche Classique — Procédurale

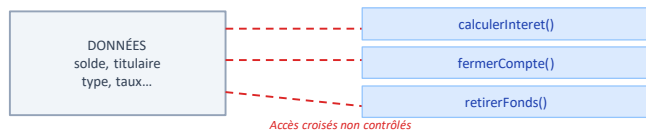
Données et fonctions séparées

PRINCIPE DE BASE

Séparation données / fonctions

Variables globales accessibles par toutes les fonctions. Aucune barrière de protection entre données et traitements.

STRUCTURE EN MÉMOIRE



PROBLÈME — APPLICATION BANCAIRE À 100 TYPES DE COMPTES

```
# Chaque fonction doit gérer tous les types
def calculerInteret(compte):
    if compte['type'] == 'epargne': ...
    elif compte['type'] == 'cheque': ...
    elif compte['type'] == 'livretA': ... # x 100 cas
```

CONSÉQUENCES

Explosion combinatoire

100 types × 20 fonctions = 2 000 blocs if/elif à maintenir et tester

Effet domino

Un bug ligne 10 peut casser la ligne 890 — aucun lien apparent, maintenance impossible

Aucune protection des données

compte['solde'] = -9999 : accessible partout. Aucun invariant garanti

VS

Approche Orientée Objet — POO

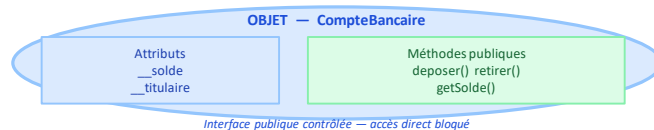
Données + comportements encapsulés dans un objet

PRINCIPE DE BASE

Regroupement données + comportements

Attributs privés + méthodes publiques réunis dans une classe. Accès contrôlé depuis l'extérieur — invariants garantis.

STRUCTURE EN MÉMOIRE



SOLUTION — CHAQUE CLASSE GÈRE SON PROPRE COMPORTEMENT

```
class CompteEpargne(CompteBancaire):
    def calculerInteret(self): return self.__solde * self.__taux

class CompteCheque(CompteBancaire):
    def calculerInteret(self): return 0 # Pas d'intérêts
```

BÉNÉFICES

Isolation du changement

101ème type → nouvelle classe uniquement. Zéro modification des 100 existantes. Zéro régression.

Polymorphisme

compte.calculerInteret() appelle automatiquement la bonne version selon le type réel de l'objet

Invariants garantis

Le setter valide : if montant < 0: raise ValueError — solde impossible à corrompre depuis l'extérieur

Programmation Orientée Objet (POO) - La Classe comme Usine à Objets

① La Classe — Définition et Rôle

Plan / Moule

Modèle abstrait définissant la structure et le comportement communs à tous les objets du même type.

Usine à Objets

Produit autant d'instances indépendantes que nécessaire.
Chaque objet est une entité distincte en mémoire.

Contrat & Implémentation

Contrat = Définit ce que fait la classe .

Implémentation = précise comment elle le fait (attributs , méth)

② Les 3 Dimensions d'un Objet — En POO : Objet = Identité + État + Comportement

1. Identité

Ce qui rend chaque objet unique.
Ex. : `compte_julien` ≠ `compte_marie`
→ Deux entités distinctes en mémoire.

2. État

Valeurs des attributs à un instant t.
Ex. : `solde = 1 500 €`, `nom = « Julien »`
→ Évolue via les méthodes.

3. Comportement

Actions que l'objet peut effectuer.
Ex. : `retirer(montant)`, `virer(dest.)`
→ Méthodes = services offerts.

Identité + État + Comportement = Un Objet logiciel complet (brique autonome réutilisable)

③ Instanciation — De la Classe à l'Objet (exemple : CompteBancaire)

CLASSE : CompteBancaire

Attributs (caractéristiques) :

- Nom (string) - Prenom (string) - Numero (int)

Méthodes (actions / services) :

- `retirer(montant)` - `deposer(montant)` - `virer(montant, destination)`

Instanciation
(new)

OBJET : Compte_Julien (instance concrète en mémoire)

Attributs (valeurs concrètes) :

- Nom = "Dupont"
- Prenom = "Julien"
- Numero = 12345

Méthodes (appels réels) :

- `retirer(500)`
- `deposer(1000)`
- `virer(200, Compte_Marie)`

④ Vocabulaire Essentiel

Classe	Objet	Attribut	Méthode	Constructeur
Plan / modèle abstrait	Instance concrète en mémoire	Caractéristique / donnée	Action / comportement	Méthode d'initialisation

Instance, Instanciation & Constructeur

Qu'est-ce qu'une instance ? Comment un objet prend-il vie à partir d'une classe ?

DÉFINITIONS FONDAMENTALES

Instance

Un **objet** créé à partir d'une **classe** via son **constructeur**. Chaque instance est indépendante : elle possède ses propres valeurs d'attributs, tout en partageant la structure définie par la classe.

Instanciation

L'opération de création d'un objet à partir d'une classe. **Instance = Objet**.

LE CONSTRUCTEUR — DÉFINITION FORMELLE

Méthode spéciale appelée automatiquement lors de la création de l'objet.

- Appelé automatiquement à la création — jamais explicitement par l'utilisateur.
- Initialise les attributs de l'instance avec les valeurs passées en paramètre.
- Peut recevoir un nombre quelconque d'arguments selon les besoins.
- En Python : méthode `__init__(self, ...)` — `self` désigne l'objet lui-même.
- En Java : méthode portant exactement le même nom que la classe, sans valeur de retour.

RÈGLE DE DÉCLARATION

Pour déclarer un constructeur, ajouter une méthode portant le même nom que la classe, suivie des paramètres souhaités et de leur implémentation.

IMPLÉMENTATION PYTHON — `__init__` & `SELF`

```
class CompteBancaire:
    def __init__(self, titulaire: str, solde: float):
        # self = référence vers l'objet en cours de création
        self.__titulaire = titulaire # attribut privé
        self.__solde = solde        # attribut privé

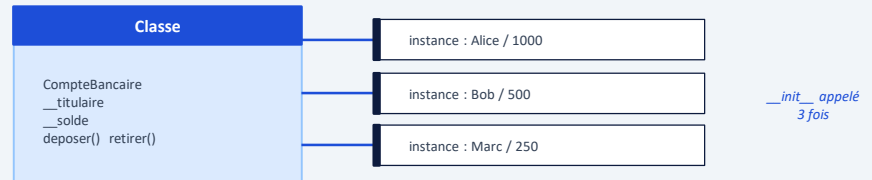
# Instanciation — __init__ appelé automatiquement
c1 = CompteBancaire('Alice', 1000) # instance 1
c2 = CompteBancaire('Bob', 500)   # instance 2
```

IMPLÉMENTATION JAVA — CONSTRUCTEUR & `THIS`

```
public class CompteBancaire {
    private String titulaire; // attribut privé
    private double solde;

    // Constructeur — même nom que la classe, sans return
    public CompteBancaire(String t, double s) {
        this.titulaire = t; // this = objet courant
        this.solde = s;
    }
}
```

UNE CLASSE → PLUSIEURS INSTANCES INDÉPENDANTES



Le Changement de Paradigme : Exécution bilangage

Python

Langage Interprété

Exécution ligne par ligne à la volée.
Pas de compilation préalable nécessaire.
Flexibilité maximale et prototypage rapide.



Java

Langage Compilé

Compilation en bytecode JVM avant exécution.
Typage statique strict, erreurs détectées
à la compilation plutôt qu'à l'exécution.



Rappel : Structures de Données essentielles en Programmation Objets

Concept	Python	Java
Chaînes (Immuables)	<code>s = "Hello"</code>	<code>String s = "Hello";</code>
Listes (Modifiables)	<code>L = [1, 2, 3]</code>	<code>List<Integer> L = new ArrayList<>();</code>
Tuples (Immuables)	<code>t = (1, "A")</code>	<code>public record MyTuple(int id, String val) {};</code>
Sets (Uniques)	<code>s = {1, 2, 3}</code>	<code>Set<Integer> s = new HashSet<>();</code>
Dicts/Maps (Clé/Valeur)	<code>d = {"k": "v"}</code>	<code>Map<String,String> d = new HashMap<>();</code>

Différence clé : Java exige des **imports** (`java.util.*`) et des types génériques strictes (`<Integer>`). Python est dynamiquement typé et indexé directement.

COMPARAISON DE SYNTAXE : PYTHON vs. JAVA (POO)

PYTHON

1. La Création (Le Constructeur)

JAVA

Python (Spécial)

C'est spécial. Tu utilises toujours le mot magique `def __init__(self, ...)`

```
self.nom = nom
```

(remplace `this`)

→ Simple, un seul mot magique

Java (Formel)

C'est rigide. Tu dois réécrire le nom de la classe `CompteBancaire(...)`

```
this.nom = "mon nom à moi"
```

→ Nom de classe obligatoire

2. Le Typage (La précision)

Python (Flexible)

Il est cool. Tu donnes juste les noms (nom, montant) sans dire si c'est du texte ou des chiffres.

```
nom = "...", montant = 500
```

→ Typage dynamique, implicite

Java (Exigeant)

Il est bavard. Tu dois préciser partout le type de donnée (String, double, void)

```
String nom = "..."
```

→ Typage statique, explicite

3. La Mise en page

Python (Visuel)

Utilise l'espace (indentation). Si ton texte est décalé vers la droite, il est « dans » la fonction. Pas besoin de symboles compliqués.

```
def __init__(self):
    action
# Indentation = structure
```

→ Structure par l'indentation

Java (Ponctué)

Utilise des accolades {} pour enfermer les blocs de code et des points-virgules ; à la fin de chaque instruction.

```
public class Main {
    ... ;
}
```

→ Structure par {} et ; obligatoires

Python · class & __init__

```
# La Classe : plan abstrait
class CompteBancaire:
    # Constructeur
    def __init__(self,
        titulaire: str,
        solde: float):
        # Attributs privés
        self.__titulaire =
            titulaire
        self.__solde = solde

    # Methodes publiques
    def deposer(self, m):
        if m > 0:
            self.__solde += m

    def retirer(self, m):
        if m <= self.__solde:
            self.__solde -= m

# Instanciation
c = CompteBancaire(
    "Julien", 1000)
c.deposer(500)
c.retirer(200)
```

Java · class & constructeur

```
// La Classe : plan abstrait
public class CompteBancaire {
    // Attributs privés
    private String titulaire;
    private double solde;

    // Constructeur
    public CompteBancaire(
        String t, double s) {
        this.titulaire = t;
        this.solde = s;
    }

    // Methodes publiques
    public void deposer(double m) {
        if (m > 0)
            this.solde += m;
    }

    public void retirer(double m) {
        if (m <= this.solde)
            this.solde -= m;
    }
}

// Instanciation
var c = new CompteBancaire(
    "Julien", 1000);
c.deposer(500);
c.retirer(200);
```


La Pyramide des Tests — Niveaux de vérification

Isolation croissante vers le bas · Vitesse d'exécution croissante vers le bas · Coût décroissant vers le bas

Lent

4

Tests Manuels

Un humain explore et tente de casser le logiciel. Très lent, non reproductible, non automatisable. Utilisé pour la validation finale.

Vitesse

3

Tests API & UI — End-to-End

Tester le produit du point de vue de l'utilisateur : cliquer un bouton, remplir un formulaire, appeler une API REST. Lent mais représentatif.

2

Tests d'Intégration & Composants

Vérifier que deux modules fonctionnent correctement ensemble. Exemple : `deposer()` interagit-elle correctement avec le solde ? Plus lent que les tests unitaires.

1

Tests Unitaires — La base (Unit Tests)

Tester chaque unité de code isolément : une méthode, une fonction. Rapide, précis sur l'origine du bug. C'est la base — doit être la plus fournie.

Python : `unittest` · `pytest` | Java : `JUnit 5` · `Maven` · `Gradle`

Rapide

RÈGLES FONDAMENTALES

Isolation (axe vertical)

Plus on descend, plus le bug est localisé précisément. Un test unitaire qui échoue pointe exactement la méthode fautive.

Vitesse (axe vertical)

Plus on descend, plus les tests sont rapides. La base (tests unitaires) doit être exécutée à chaque sauvegarde.

Règle industrielle — Pyramide, pas losange

Majorité des tests = unitaires (base large). Peu de tests E2E (sommet étroit). Un projet inversé est fragile et coûteux.

EXEMPLE — TEST UNITAIRE PYTHON (PYTEST)

```
# Test unitaire : méthode depoter()
def test_depot_valide():
    """deposer 500€ sur solde initial 1000€ → 1500€"""
    c = CompteBancaire('Alice', 1000) (Arrange)
    c.deposer(500) (Agir)
    assert c.get_solde() == 1500 (Assert = verification)

# GREEN — test passe · isolation garantie
```

EXEMPLE — TEST UNITAIRE JAVA (JUNIT 5)

```
@Test
void testDepotValide() {
    var c = new CompteBancaire("Alice", 1000); (Arrange)
    c.deposer(500); (Agir)
    assertEquals(1500, c.getSolde()); (Assert)
```

Principe des Tests Unitaires Py test & JUnit 5: Pattern AAA

1. **Arrange** : Tu crée un objet de test (`c = CompteBancaire('Alice', 1000)`)
2. **Agir** : Tu fais l'action que tu veux tester, le dépôt (`c.deposer(500)`)
3. **Assert (clé)** : Tu vérifie si le solde (`assert c.get_solde() == 1500`)
4. **Si c'est vrai** ($1000 + 500 = 1500$) : le test passe au vert (**Green**)
5. **Si c'est faux** (par exemple si ton code affiche 1000 au lieu de 1500) Le test est **rouge**

Approche TDD — Test-Driven Development : Clé de l'ingénieur Logicielle

Standard industriel de qualité — écrire le test AVANT le code, puis itérer : RED → GREEN → REFACTOR

PRINCIPE FONDAMENTAL

Avoir une idée précise du comportement attendu AVANT d'écrire la fonctionnalité. En POO : écrire le test qui valide une méthode AVANT d'écrire le code de cette méthode. Le test définit l'interface et le comportement attendu.

1 RED — Écrire le test

Le test doit échouer

- Écrire un test pour la fonctionnalité AVANT d'écrire le code.
- Le test DOIT échouer car la fonctionnalité n'existe pas encore.
- Pour chaque fonctionnalité : commencer par son test.
- Le test définit l'interface et le comportement attendu.

```
def test_depot():
    c = CompteBancaire()
    c.deposer(100)
    assert c.solde == 100

# FAIL - méthode inexistante
```

2 GREEN — Faire passer le test

Code minimal uniquement

- Écrire le code MINIMAL pour que le test passe.
- Ne pas chercher la perfection — juste faire passer le test.
- Développer jusqu'à ce que le test soit en succès.
- Chaque itération = un test, puis le code qui le valide.

```
class CompteBancaire:
    def __init__(self):
        self.solde = 0

    def depoter(self, m):
        self.solde += m

# PASS - test validé !
```

3 REFACTOR — Améliorer le code

Sans casser les tests

- Nettoyer et restructurer le code sans modifier son comportement.
- Améliorer lisibilité, supprimer duplication, renforcer architecture.
- Les tests existants garantissent l'absence de régression.
- Puis recommencer : RED → GREEN → REFACTOR en boucle.

```
def depoter(self, montant: float):
    if montant <= 0:
        raise ValueError(
            "Montant invalide"
        )
    self.__solde += montant

# Tests PASS - invariants ok
```

ITÉRATIONS SUCCESSIVES → DÉPLOIEMENT FINAL

L'application se construit par itérations RED → GREEN → REFACTOR jusqu'à l'aboutissement du projet. Chaque itération produit du code testé, fonctionnel et propre.

TDD Appliqué — Use Case : CompteBancaire

Application complète du cycle Red → Green → Refactor sur le cas fil rouge Python

1 Phase RED — Écrire le test AVANT la classe

Objectif : écrire le test de `deposer()` AVANT que `CompteBancaire` n'existe. Le test DOIT échouer — la méthode n'existe pas encore. C'est volontaire.

CODE DU TEST (PYTEST) — PHASE RED

```
# Aucune classe n'existe encore
def test_depot_valide():
    """Déposer 100€ sur un compte vide → solde = 100"""
    compte = CompteBancaire()  # pas encore créée
    compte.deposer(100)
    assert compte.solde == 100

# FAIL — NameError: name 'CompteBancaire' is not defined
```

2 Phase GREEN — Code minimal pour passer le test

Objectif : écrire le code minimal de `CompteBancaire` pour que le test passe. Rien de plus — pas de validation, pas d'encapsulation encore.

CODE MINIMAL DE LA CLASSE — PHASE GREEN

```
class CompteBancaire:
    def __init__(self):
        self.solde = 0  # minimal

    def deposter(self, montant):
        self.solde += montant  # minimal

# PASS — Le test est vert !
```

3 Extension — Cycle répété sur la logique du retrait

Même démarche pour chaque fonctionnalité. Le test définit l'interface et le comportement avant toute implémentation.

TEST RETRAIT — PHASE RED

```
def test_retrait_valide():
    """Retrait 50€ depuis 100€ → solde = 50"""
    c = CompteBancaire()
    c.deposer(100)
    c.retirer(50)
    assert c.solde == 50
```

CODE RETRAIT MINIMAL — PHASE GREEN

```
def retirer(self, m):
    self.solde -= m

# PASS — solde = 50
```

REFACTOR — AJOUT INVARIANTS & ENCAPSULATION

```
class CompteBancaire:
    def __init__(self):
        self.__solde = 0  # privé
    def deposter(self, m: float):
        if m <= 0: raise ValueError
        self.__solde += m
    def retirer(self, m: float):
        if m > self.__solde: raise ValueError
        self.__solde -= m

# Tests PASS — invariants garantis
```

Principe clé — Le test définit l'interface ET le comportement attendu.

Objectif : concevoir des classes dont chaque méthode est couverte par au moins un test automatisé.

Les 3 Piliers de la Modélisation Objet Avancée & Use cases

01 Héritage



Réutilisation et Architecture

Définir une classe dérivée (enfant) à partir d'une classe de base (parent) déjà existante.

02 Encapsulation



Sécurité et Accès

Protéger les données internes en interdisant l'accès direct depuis l'extérieur de la classe.

03 Polymorphisme

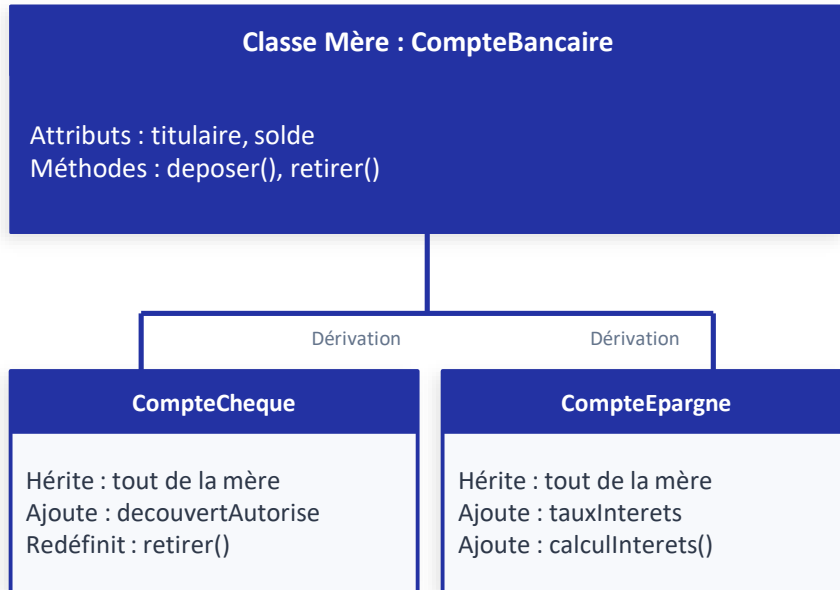


Flexibilité et Interfaces

Une même méthode peut avoir des comportements différents selon l'objet sur lequel elle est appelée.

Pilier 1 : Héritage (Réutilisation)

Définir une classe dérivée (enfant) à partir d'une classe de base (parent) déjà existante.



Concepts Clés du Héritage

Python : `class Enfant(Parent):`

Java : `class Enfant extends Parent {}`

super() : Appel au constructeur de la classe mère

Override : Redéfinir une méthode héritée

Bénéfice : Éviter la duplication de code (DRY)

Héritage : Implémentation Code

Python · Parenthèses & super()

```
def __init__(self, titulaire,
              solde, dec_max):
    super().__init__(titulaire, solde)
    self.decouvert = dec_max

def retirer(self, montant):
    if self.__solde + self.decouvert
        >= montant:
        self.__solde -= montant
```

Java · extends & super()

```
public class CompteCheque
    extends CompteBancaire {

    private double decouvert;

    public CompteCheque(
        String t, double s, double d) {
        super(t, s);
        this.decouvert = d;
    }
}
```

Différence : Python utilise les **parenthèses (Parent)** — Java utilise le mot-clé **extends Parent**. Les deux utilisent **super()** pour appeler le constructeur parent

Pilier 2 : Encapsulation (Sécurité)

Regrouper les attributs et les méthodes pour créer une nouvelle classe.

La Zone Privée (Le Coffre)

```
self.__titulaire (private)
self.__solde      (private)
```

Inaccessible de l'extérieur (Accès direct bloqué)

✓ La Zone Publique (L'Écran ATM)

```
getSolde()      → lecture sécurisée
deposer(montant) → validation puis modification
```

Analogie ATM : On ne touche pas directement aux billets (attributs privés). On utilise l'écran et les boutons (**méthodes publiques**) qui contrôlent et valident chaque action.

Pourquoi Encapsuler ?



Sécurité

Restreint l'accès direct aux données. Impossible de corrompre l'état depuis l'extérieur.



Validation

Le Setter vérifie la valeur avant modification (ex: solde ne peut pas être négatif).



Maintenabilité

Changer l'implémentation interne sans modifier l'API publique.

Le Mécanisme : Getters & Setters

Getter (Accesseur)

Retrouve une valeur ou une chaîne sans la modifier.
Retourne simplement l'attribut privé.

```
# Python
@property
def get_solde(self):
    return self.__solde

// Java
public double getSolde() {
    return this.solde;
}
```

Setter (Mutateur)

Ne renvoie rien, mais effectue des vérifications logiques
avant d'autoriser la modification.

```
# Python
def set_solde(self, montant):
    if montant > 0:
        self.__solde += montant

// Java
public void setSolde(double m) {
    if (m > 0) this.solde += m;
}
```

Getter

→ lire sans modifier

Setter

→ vérifier puis modifier

Encapsulation : Implémentation Code

Python · Convention des Underscores

```
class CompteBancaire:
    def __init__(self, titulaire, solde):
        self.__titulaire = titulaire # privé
        self.__solde = solde        # privé

    # Getter
    @property
    def get_titulaire(self):
        return self.__titulaire

    # Setter
    def deposer(self, montant):
        if montant > 0:
            self.__solde += montant

# X compte.__solde = 50000 → AttributeError
```

Java · Mot-clé private

```
public class CompteBancaire {
    private String titulaire; // privé
    private double solde;    // privé

    public CompteBancaire(String t, double s){
        this.titulaire = t;
        this.solde = s;
    }

    // Getter
    public String getTitulaire() {
        return titulaire;
    }

    // Setter
    // X compte.solde = 50000 → erreur compile
}
```

Pilier 3 · Polymorphisme (Flexibilité)

Pilier 3 : Polymorphisme (Flexibilité)

"Une même méthode, plusieurs comportements."

L'appel générique : `compte.retirer(1200)`

Objet : CompteCheque

Comportement redéfini :

Applique la logique avec découvert autorisé.

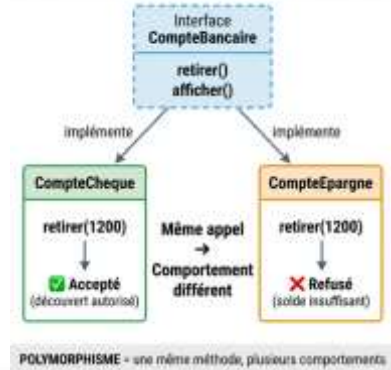
✓ Résultat : Accepté

Objet : CompteEpargne

Comportement redéfini :

Applique la logique stricte sans découvert.

✗ Résultat : Refusé (solde insuffisant)



Puissance : Un seul appel générique permet d'exécuter automatiquement le bon comportement selon le type réel de l'objet. → **code flexible et extensible.**

Polymorphisme : Implémentation Code

Python · "Duck Typing"

```
# Pas besoin de lien d'héritage strict.  
# Si l'objet a la méthode → ça marche.  
  
class CompteCheque(CompteBancaire):  
    def retirer(self, montant):  
        if self.__solde + self.decouvert  
            >= montant:  
            self.__solde -= montant  
  
# Appel polymorphe :  
for compte in comptes:  
    compte.retirer(1200) # dispatche auto
```

Python : Duck Typing — vérification à l'exécution.

Java · Interfaces & @Override

```
// Interface définissant le contrat  
interface ICompte {  
    void retirer(double montant);  
}  
  
class CompteCheque implements ICompte {  
    @Override  
    public void retirer(double montant) {  
        // logique avec découvert  
    }  
}  
  
// Appel polymorphe :  
ICompte c = new CompteCheque(...);  
c.retirer(1200);
```

Java : Interfaces & @Override — vérification à la compilation.

Synthèse : Les Avantages de l'Architecture POO



Réutilisation de Code

via l'Héritage

Les classes peuvent être héritées et étendues, évitant la répétition et accélérant le développement. Principe DRY (Don't Repeat Yourself).



Sécurité

via l'Encapsulation

Restreint l'accès direct aux données. Garantit que les informations sont manipulées correctement et protégées des éléments extérieurs.



Maintenabilité

via la Modularité

En divisant le code en classes, il devient plus facile de localiser, isoler et corriger les bugs sans affecter le reste du système.



Modularité & Abstraction

via le Polymorphisme

Cache les détails complexes d'implémentation derrière une interface publique claire. Permet une mise à jour granulaire des modules.

Python vs Java : Tableau Récapitulatif POO

Concept	Python	Java
Définir une classe	<code>class MaClasse:</code>	<code>public class MaClasse {}</code>
Constructeur	<code>def __init__(self):</code>	<code>public MaClasse() {}</code>
Attribut privé	<code>self.__attribut</code>	<code>private String attribut;</code>
Héritage	<code>class E(Parent):</code>	<code>class E extends Parent {}</code>
Appel parent	<code>super().__init__(...)</code>	<code>super(...)</code>
Override méthode	<code>def methode(self):</code>	<code>@Override public void methode(){}</code>
Encapsulation	<code>self.__secret</code> (Double underscore)	<code>private String secret;</code>
Polymorphisme	Duck Typing	Interface / implements

Fin du Module 1

Travaux Pratiques (TP) : Python & Java

Objectif : Implémenter vos propres classes, sécuriser vos attributs, et tester vos hiérarchies d'héritage.

Ressources Complémentaires

🔗 <https://koor.fr/>

Tutoriels, supports de cours,
exemples de codes et test
de connaissance Java & Python , guide technique .

01 Héritage

02 Encapsulation

03 Polymorphisme